

UGANDA MARTYRS UNIVERSITY

Faculty of Science | Department of Computer Science & Information Systems

PROJECT REPORT

UMU Event Management System

CSC 2202 — Web-based System Programming

Course Unit	CSC 2202 — Web-based System Programming
Year of Study	Second Year — BSc IT / BScEd
Semester	Semester 2, 2025/2026
Project Topic	Option B — Event Management System
ID	REG NO
1	2024-B072-33329
2	2024-B221-31814
3	2024-B072-33330
4	2024-B201-31156
5	2024-B072-30183

System Description

The UMU Event Management System is a dynamic, database-driven web application developed as the End-of-Semester project for CSC 2202 — Web-based System Programming at Uganda Martyrs University. The system enables the management, discovery, and attendance of university events through a structured role-based platform

Purpose

The system provides a centralised platform where students can discover approved university events, RSVP to events of interest, and post comments. Verified users and administrators can create events with poster images. The system enforces approval workflows, daily reminder notifications, and role-based access controls to maintain quality and security.

Key Functional Features

Student registration and secure login using bcrypt password hashing
Role-based access: Student · Verified User · Administrator
Browse, search, and filter approved events by keyword, category, date range, and free/paid status
RSVP to events with duplicate prevention and capacity enforcement
Full comment CRUD on event detail pages (post, edit, delete)
Event creation with poster image upload (JPG/PNG/WebP, max 5MB)
Verified user events go through admin approval workflow before publishing
Daily automated reminder notifications sent on login for upcoming RSVPd events
Admin panel: approve/reject events, manage users, send bulk or targeted notifications
Admin can verify user accounts to grant event-creation rights
UMU branding: Black · Red (#C8102E) · Yellow (#F5C518) throughout

Technology Stack

Layer	Technology Used
Backend	PHP
Database	MySQL
Frontend	HTML, CSS

Database Structure

The system uses a relational MySQL database named umu_events. The schema consists of seven normalised tables designed to support all functional requirements including role management, event lifecycle, attendance tracking, commenting, and notifications.

Entity-Relationship Overview

The central entity is events, which relates to users (via creator_id), categories, rsmps, comments, and notifications. The users table manages all three roles. Below is the description of each table.

users

id	INT PK AI	Auto-incremented primary key
reg_number	VARCHAR(30)	Unique university registration number used for login
full_name	VARCHAR(120)	Student or staff full name
email	VARCHAR(120)	Unique email address
password	VARCHAR(255)	bcrypt-hashed password
role	ENUM	'student' 'verified' 'admin'
is_active	TINYINT(1)	Account active status (admin can deactivate)
created_at	TIMESTAMP	Account registration timestamp

categories

id	INT PK AI	Auto-incremented primary key
name	VARCHAR(60)	Category name e.g. Academic, Sports, Cultural
icon	VARCHAR(10)	Emoji icon for display

events

id	INT PK AI	Primary key
creator_id	INT FK	Foreign key → users.id
category_id	INT FK	Foreign key → categories.id
title	VARCHAR(200)	Event title
description	TEXT	Full event description
location	VARCHAR(200)	Venue name

event_date	DATETIME	Date and time of the event
is_free	TINYINT(1)	1 = free, 0 = paid
price	DECIMAL(10,2)	Ticket price in UGX (0.00 if free)
capacity	INT NULL	Maximum attendees; NULL = unlimited
poster	VARCHAR(255)	Uploaded poster filename in uploads/posters/
status	ENUM	'pending' 'approved' 'rejected'
rejection_reason	VARCHAR(255)	Optional rejection feedback from admin
created_at	TIMESTAMP	Record creation timestamp

rsvps

id	INT PK AI	Primary key
user_id	INT FK	Foreign key → users.id
event_id	INT FK	Foreign key → events.id
rsvp_at	TIMESTAMP	Timestamp of RSVP registration
UNIQUE KEY	(user_id, event_id)	Prevents duplicate RSVPs

comments

id	INT PK AI	Primary key
event_id	INT FK	Foreign key → events.id (CASCADE delete)
user_id	INT FK	Foreign key → users.id
body	TEXT	Comment content
created_at	TIMESTAMP	Time comment was posted
updated_at	TIMESTAMP	Auto-updated on edit; used to show '(edited)' label

notifications

id	INT PK AI	Primary key
user_id	INT FK	Recipient user
title	VARCHAR(200)	Short notification headline

body	TEXT	Full notification message
type	ENUM	'reminder' 'approval' 'rejection' 'comment' 'system'
is_read	TINYINT(1)	Read status; unread shown with red badge
event_id	INT FK NULL	Optional link to related event

Functional Requirements Coverage

The table below maps each of the ten functional requirements specified in the assessment brief to the corresponding implementation in the UMU Event Management System.

	Requirement	How It Is Met
1	User Registration & Login	index.php handles both. Registration validates all fields server-side, checks for duplicate reg numbers/emails, and stores bcrypt-hashed passwords. Login verifies against the hash and starts a PHP session.
2	Logout & Session Control	logout.php calls session_unset() and session_destroy(). Every protected page calls requireLogin(), requireAdmin(), or requireCanCreate() which redirect unauthenticated users back to index.php.
3	Data Entry through Forms	Events are created via create_event.php with poster upload. RSVPs submitted via events.php and event_detail.php. Comments posted on event_detail.php. All data stored in MySQL and persists across refreshes.
4	Input Validation with Feedback	All forms validate server-side in PHP. Required fields, email format, password length, date/time presence, and price logic are all checked. Errors are shown as styled alert boxes. Invalid data is never saved.
5	Dynamic Data Display	All events, RSVPs, comments, and notifications are fetched from the MySQL database using prepared statements. No content is hard-coded in HTML. Pages reflect real-time database state.
6	Search / Filter	events.php provides a combined search bar (keyword across title/description/location), category dropdown, date-from/date-to range pickers, and a free-only checkbox. Results update dynamically from the database.
7	Editing Existing Records	Events: create_event.php?edit=ID loads existing data into the form for update. Comments: edit link on event_detail.php shows inline edit form. Both use UPDATE SQL with ownership checks.

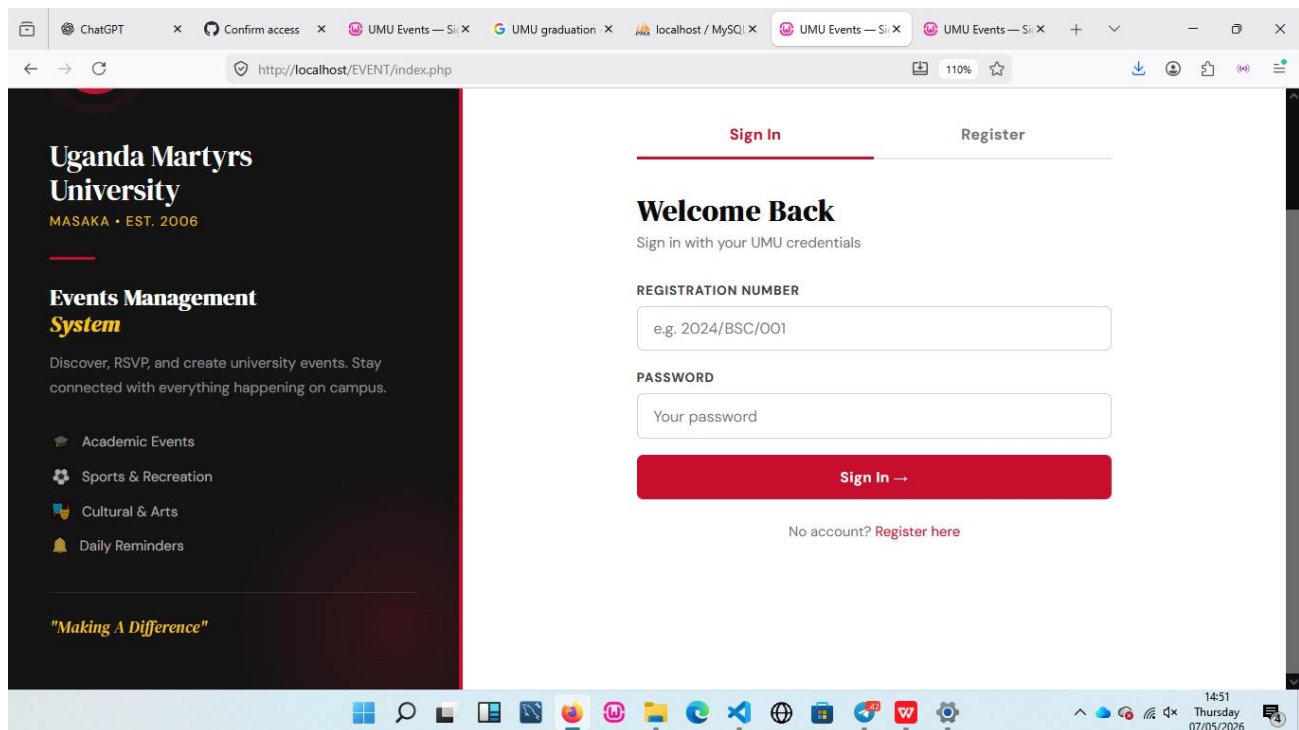
8	Deleting Records	Admin can delete events (admin/events.php) which also removes the poster file. Comment owners and admins can delete comments via DELETE SQL. Students can cancel RSVPs. Admin can remove users.
9	Role-Based Access	Three roles: student (browse/RSVP/comment only), verified (can create events, pending approval), admin (full CRUD + approve/reject events + verify users + send notifications). PHP guards enforce this on every page.
10	Persistent Data Storage	All data stored in MySQL umu_events database. Server restarts and browser refreshes do not affect data. Uploaded posters stored in uploads/posters/ folder on the server filesystem.

Screenshots of key screens

The following sections describe the five key screens of the UMU Event Management System. Screenshots should be inserted in place of the placeholder boxes below.

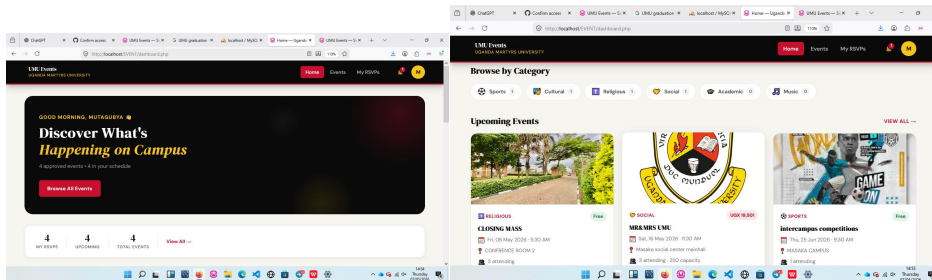
Login & Registration Page

The authentication page features a two-column layout: the left panel displays the UMU branding (logo placeholder, university name, tagline, and feature list) styled in the UMU black and red colour scheme. The right panel contains a tabbed form that switches between Sign In and Register. The registration form collects full name, registration number, email, and password with confirmation. All fields are validated server-side before processing.



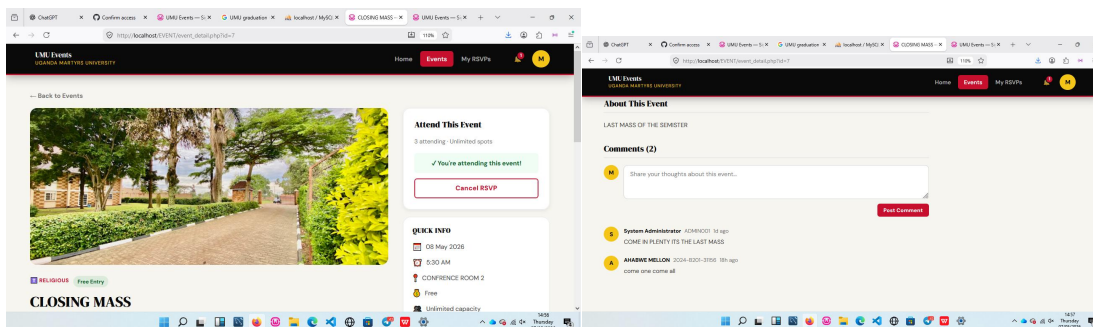
Student Dashboard

The student dashboard displays a hero banner with a personalised greeting and event count statistics. Below it, a category filter strip allows quick filtering. Upcoming events are shown in a responsive card grid, each card displaying the event poster (or a coloured placeholder), category badge, free/paid status, date, location, and an RSVP button. Students who have already RSVPd see a green confirmation indicator instead.



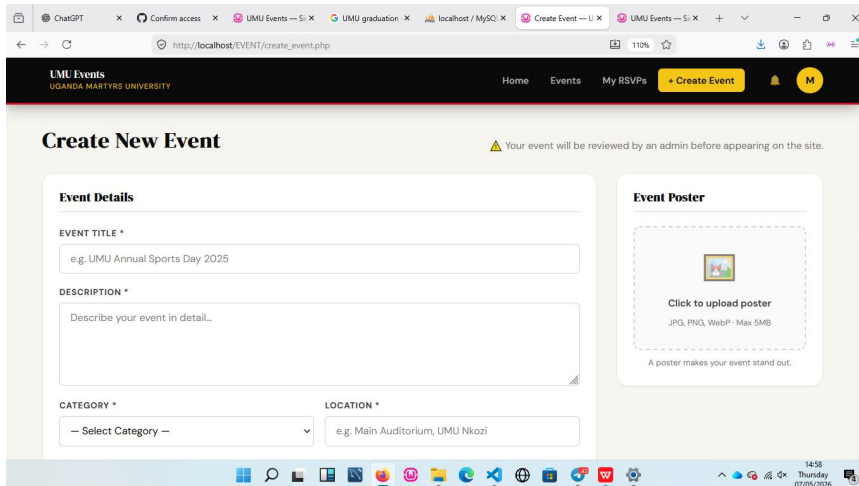
Event Detail Page with Comments

The event detail page uses a two-column layout. The main column shows the event poster, status banner (if pending/rejected), event title, creator information, an info grid (date, location, attendance, comment count), full description, and the comment section. The comment section includes a post form and lists all existing comments with edit and delete controls for the owner or admin. The sidebar contains the RSVP card with a capacity progress bar.



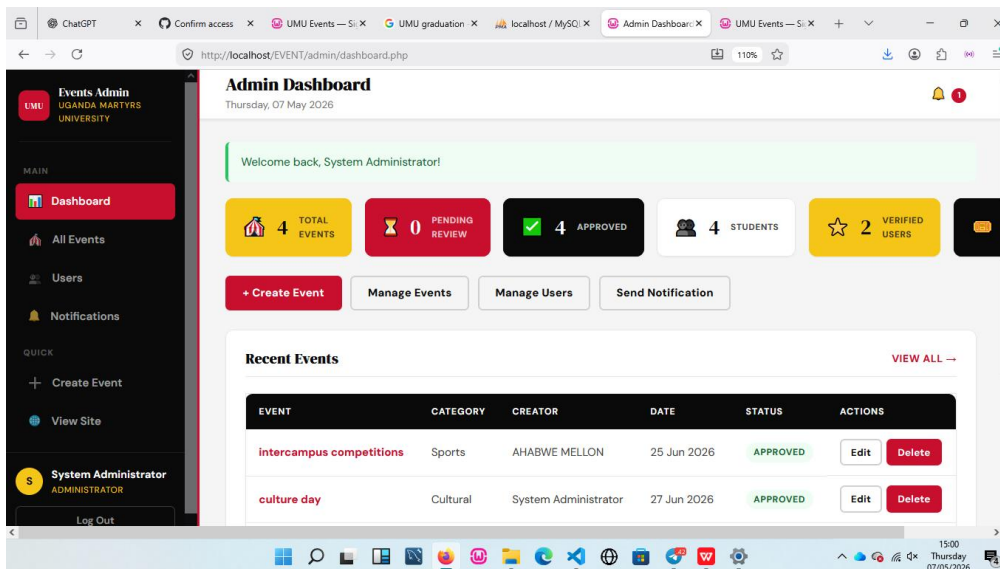
Create Event Page

The create event page is available to admin and verified users only. It presents a two-column form layout: the left panel collects event title, description, category, location, date, time, capacity, and pricing (with a toggle switch for free/paid). The right panel provides a poster upload area with a live image preview before submission. Admin-created events publish immediately; verified user events are flagged as pending and trigger a notification to all administrators.



Admin Dashboard

The admin panel uses a sidebar navigation layout with the UMU logo slot at the top. The dashboard displays six statistics cards (total events, pending, approved, students, verified users, RSVPs) followed by quick action buttons. Below that, a highlighted table lists all pending events requiring approval, with approve and reject action buttons. A recent events table shows the latest activity with status badges.



Challenges Faced and Solutions

The following section reflects honestly on the key technical and design challenges encountered during the development of the UMU Event Management System.

Implementing Role-Based Access Without a Framework

One of the primary challenges was building a secure role-based access system in plain PHP without using any MVC framework. Ensuring that students could not access admin pages, and that

unverified students could not create events, required careful placement of session guard functions at the top of every protected page.

Solution: Centralised guard functions (requireLogin, requireAdmin, requireCanCreate) were written in includes/auth.php and called at the very top of each page before any output. This ensured consistent enforcement across all pages.

Preventing Duplicate RSVPs

The RSVP feature required that a student cannot register for the same event more than once, even if they tried to submit the form multiple times quickly. A simple PHP check on the previous request was not sufficient.

Solution: A UNIQUE KEY constraint was added to the rsmps table on the combination of (user_id, event_id). This means the database itself rejects any duplicate RSVP at the storage level, providing a second layer of protection beyond the PHP check. A clear error message is shown to the user if a duplicate is attempted

File Upload Security and Validation

Allowing users to upload event poster images introduced security risks, including attempts to upload executable files disguised as images, and issues with file size limits.

Solution: The uploadPoster() function in includes/db.php uses PHP's finfo extension to check the actual MIME type of the uploaded file against a whitelist (image/jpeg, image/png, image/webp, image/gif), rather than relying on the file extension which can be spoofed. File size is capped at 5MB. Uploaded files are saved with randomly generated names using uniqid() to prevent overwriting and directory traversal.

Daily Reminder Notifications

The assessment required that the system send daily reminders to students about events they have RSVPd for. PHP does not natively support background tasks or scheduled jobs without a cron job, which is not available in a basic WAMP development setup.

Solution: The send Daily Reminders() function was placed inside includes/auth.php and called immediately after a successful login. The function queries for approved events within the next 3 days that the user has RSVPd for, then checks whether a reminder notification already exists for that user-event combination on the current date (DATE(created_at) = CURDATE()). This prevents duplicate reminders from being generated if the user logs in multiple times in one day.

Conclusion

The UMU Event Management System successfully demonstrates all ten functional capabilities required by the assessment brief. The system was built using core PHP, MySQL, HTML5, and CSS without reliance on external frameworks, demonstrating a solid understanding of web-based system programming fundamentals.

The project goes beyond the minimum requirements by implementing three user roles, poster image uploads, an approval workflow with rejection feedback, daily automated reminder notifications, bulk notification sending from the admin panel, a

The code base follows a clean separation of concerns: PHP logic handles and capacity tracking with visual progress indicators.all data processing and access control, MySQL provides persistent relational data storage, and CSS (kept strictly separate from PHP) handles all visual presentation in UMU brand colors. The system is ready to be run on any standard WAMP local server environment by following the setup instructions in the README.md file.

Refrences

Welling, L. and Thomson, L., 2017. *PHP and MySQL Web Development*. 5th ed. Indianapolis: Addison-Wesley Professional.

Silberschatz, A., Korth, H.F. and Sudarshan, S., 2019. *Database System Concepts*. 7th ed. New York: McGraw-Hill Education.

Mozilla Developer Network (MDN), 2026. *MDN Web Docs*. [online] Available at: <https://developer.mozilla.org/> [Accessed 4 May 2026].

PHP Documentation, 2026. *PHP Manual*. [online] Available at: <https://www.php.net/docs.php> [Accessed 4 May 2026].

Oracle Corporation, 2026. *MySQL Documentation*. [online] Available at: <https://dev.mysql.com/doc/> [Accessed 4 May 2026].